Thực hành Chương 3 — Value-Based Reinforcement Learning

Mục tiêu thực hành chương 3:

- Hiểu bản chất bootstrapping
- Phân biệt TD(0), n-step, Monte Carlo
- Hiểu vì sao cần function approximation
- Làm nền tảng cho Deep Q-Learning
- Sẵn sàng học DRL (Chương 5)

Sinh viên sẽ lập trình ba nhóm thuật toán cốt lõi:

1. Temporal Difference Learning
2. n-step Q-learning
3. Q-function Approximation với Neural Network

Môi trường sử dụng:

- Gymnasium
- Môi trường: CartPole
- Framework Deep Learning: PyTorch

3.1 — Temporal Difference Learning (TD Learning)

Mục tiêu

- Học giá trị trạng thái từ trải nghiệm từng bước
- Không cần mô hình môi trường
- Cập nhật online

Thuật toán TD(0)

Cập nhật giá trị:

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$$

Thành phần:

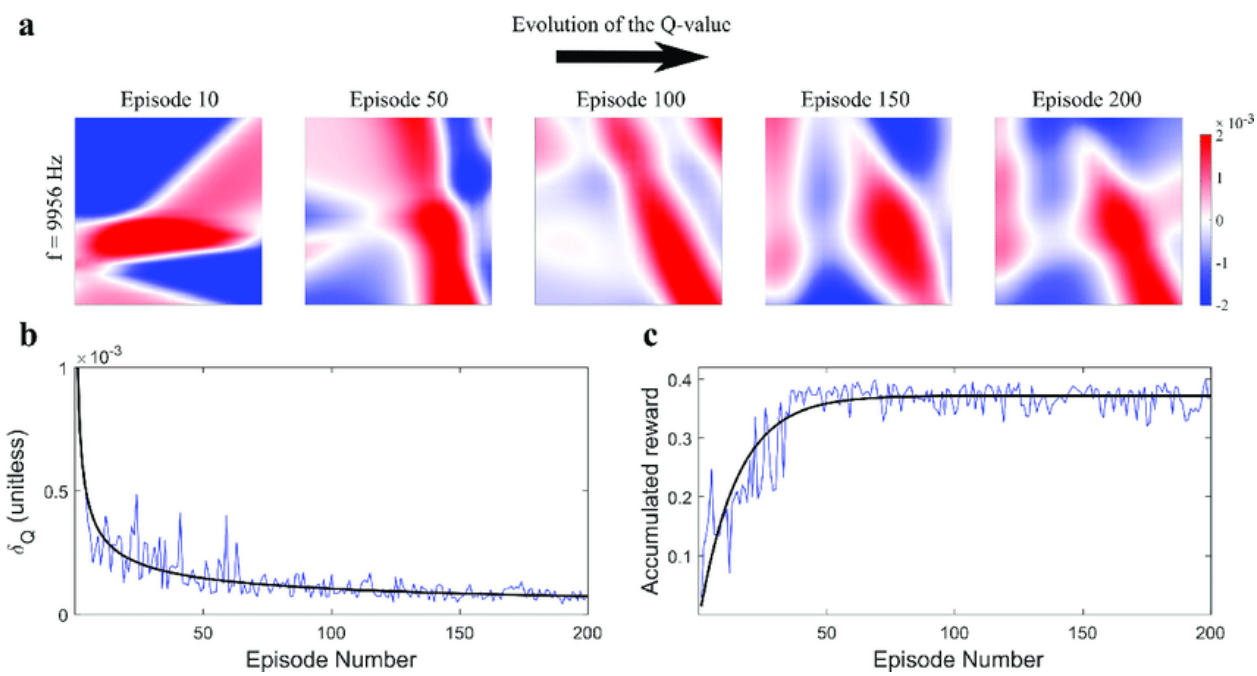
- α — learning rate
- γ — discount factor
- r — reward
- V(s) — giá trị trạng thái hiện tại

Quy trình

- Quan sát trạng thái s
- Chọn hành động a
- Nhận r, s'
- Cập nhật V(s)
- Lặp cho đến khi kết thúc episode

Yêu cầu thực hiện:

1. Cài đặt bảng giá trị V(s)
2. Viết hàm cập nhật TD(0)
3. So sánh với Monte Carlo Prediction
4. Vẽ đồ thị sai số theo episode



Câu hỏi cần trả lời:

1. Bootstrapping trong TD Learning là gì? Vì sao TD không cần đợi đến cuối episode?
2. So sánh cách cập nhật của TD(0) với Monte Carlo Prediction.
3. TD(0) có bias và variance như thế nào so với Monte Carlo?
4. Vì sao TD Learning phù hợp với môi trường online hơn Monte Carlo?
5. Giải thích ý nghĩa từng thành phần trong công thức cập nhật TD(0).
6. Nếu learning rate quá lớn hoặc quá nhỏ thì ảnh hưởng gì đến hội tụ?
7. Khi gamma tiến gần 1, giá trị trạng thái thay đổi như thế nào?
8. Cài đặt bảng giá trị V(s) cho môi trường CartPole như thế nào khi state là continuous?
9. Viết hàm update TD(0) và giải thích từng dòng code.

10. Làm thế nào để discretize state trong CartPole?
11. So sánh TD(0) với Monte Carlo theo: tốc độ hội tụ, độ ổn định
12. Khi số episode tăng, sai số giữa V(s) và giá trị thực thay đổi ra sao?
13. Vẽ learning curve và giải thích xu hướng
14. TD(0) có thể áp dụng trực tiếp cho action-value không?
15. Nếu reward rất thưa (sparse), TD(0) gặp vấn đề gì?

3.2 — n-step Q-learning

Mục tiêu:

- Kết hợp ưu điểm của Monte Carlo và TD
- Học từ nhiều bước phần thưởng
- Giảm phương sai so với MC
- Lan truyền reward nhanh hơn TD(0)

Công thức n-step return

$$G_t^{(n)} = r_t + \gamma r_{t+1} + \ldots + \gamma^{n-1} r_{t+n-1} + \gamma^n \max_a Q(s_{t+n}, a)$$

Cập nhật:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[G_t^{(n)} - Q(s,a)]$$

Quy trình thuật toán

1. Thu thập n bước trải nghiệm
2. Tính n-step return
3. Cập nhật Q-table
4. Dịch cửa sổ học sang bước tiếp theo

Yêu cầu thực hiện:

1. Cài đặt n-step buffer
2. So sánh n = 1, 3, 5
3. So sánh với TD(0)
4. Vẽ learning curve

Câu hỏi cần trả lời:

1. Ý tưởng chính của n-step learning là gì?
2. n-step return khác gì so với return của TD(0) và Monte Carlo?
3. Khi n tăng, bias và variance thay đổi như thế nào?

4. Giải thích công thức n-step return trong Q-learning.
5. Vai trò của max Q trong cập nhật là gì?
6. Tại sao n-step giúp lan truyền reward nhanh hơn TD(0)?
7. Thiết kế cấu trúc n-step buffer gồm những thành phần nào?
8. Làm thế nào để xử lý khi episode kết thúc trước khi đủ n bước?
9. Viết hàm tính n-step return từ buffer
10. So sánh với các giá trị n = 1, 3, 5: tốc độ học, độ ổn định
11. Khi n quá lớn, điều gì xảy ra?
12. So sánh n-step Q-learning với TD(0)
13. Khi nào nên chọn n nhỏ? Khi nào nên chọn n lớn?
14. n-step có phù hợp với môi trường dài episode không?

3.3 — Q-function Approximation với Neural Network

Mục tiêu

- Hạn chế của Q-table khi state lớn/liên tục
- Học xấp xỉ hàm Q bằng Deep Learning
- Nền tảng của Deep Q-Network

Ý tưởng chính

Thay vì lưu bảng:

$$Q(s,a) \Rightarrow Q_\theta(s,a)$$

Trong đó:

- θ là tham số mạng neural
- Input: trạng thái s
- Output: giá trị Q cho mọi hành động

Kiến trúc mạng cơ bản

- Input layer: vector trạng thái
- Hidden layers: Fully-connected + ReLU
- Output layer: số node = số action

Loss function:

$$L = (r + \gamma \max_a Q(s',a) - Q(s,a))^2$$

Yêu cầu thực hiện:

1. Xây dựng mạng neural bằng PyTorch
2. Cài đặt replay buffer đơn giản
3. Huấn luyện mạng dự đoán Q-value
4. So sánh với Q-table
5. Vẽ đồ thị loss và reward

Câu hỏi cần trả lời:

1. Vì sao Q-table không phù hợp với state liên tục?
2. Ý tưởng của function approximation là gì?
3. Sự khác biệt giữa Q-table và Q-network
4. Giải thích kiến trúc mạng neural dùng để xấp xỉ Q(s, a)
5. Vì sao output layer có số node bằng số action?
6. Vai trò của hàm ReLU trong mạng
7. Giải thích hàm loss trong Q-learning với neural network
8. Vì sao target có dạng max Q(s’, a)?
9. Điều gì xảy ra nếu target thay đổi quá nhanh? So sánh Q-network với Q-table về: khả năng tổng quát hóa, tốc độ học .
10. Quan sát đồ thị loss: khi nào model học tốt?
11. Quan sát reward: khi nào agent đạt policy tốt?